FEAT-ZD-48600 · DE-01 Checking the coding prerequisites ■ · Repository (and Databricks) availability and write access

Data Engineering agent · item task.repo_access · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.repo_access.md · approved by Data engineer sim.data_engineer

task.repo_access: FEAT-ZD-48600

DE-01 "Checking the coding prerequisites" (attempt 1) for the Data Engineering lane. The step checks Databricks access, whether the repository is available, whether a repository must be created, and write access. The outcome: the repository the Data Engineering tasks need is available with write access. Databricks is not applicable.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-01 Checking the coding prerequisites (attempt 1)	record:brief
Trigger	Task creation (feat.tasks from DES-04)	record:brief; feat.tasks
Gate	No human gate: automated checks, the independent judge and sampled human evaluation	record:brief
Inputs	feat.tasks, org.engineering_stack	record:brief

1. What the Data Engineering lane needs

Task	Story	Repository	Files in budget	Source
TSK-01	STORY-01	ap_pipeline	`src/ap_pipeline/transform.py`; `src/ap_pipeline/__init__.py` (version step only); `tests/test_pipeline.py`	feat.tasks (section 3, TSK-01)
TSK-02	STORY-02	ap_pipeline	New `src/ap_pipeline/delays.py`; new `tests/test_delays.py`	feat.tasks (section 3, TSK-02)

Both Data Engineering tasks are in **one repository, ap_pipeline**. Neither task names Databricks, a database, a workspace or a new repository (feat.tasks sections 1 and 3).

2. Prerequisite checks

#	Check	Result	Evidence	Source
P-1	Databricks access	**Not applicable.** No repository uses Databricks or a database, and ap_pipeline uses no Databricks or other workspace	org.engineering_stack: "No repository uses Databricks or a database"; tool view: "No Databricks or other workspace is used by this repository"	org.engineering_stack; tool:repo-access

#	Check	Result	Evidence	Source
P-2	Repository available	**Pass.** ap_pipeline is checked out locally. It has `src/ap_pipeline/` (`__init__.py`, `dq.py`, `ingest.py`, `pipeline.py`, `schedule.json`, `transform.py`), `tests/` (`__init__.py`, `test_pipeline.py`), `ci.py`, `data/sample_feed.csv`, `README.md` and `CLAUDE.md`	Checkout file listing; tool view	tool:repo-access; file:ap_pipeline/README.md
P-3	Repository creation needed	**No.** TSK-01 and TSK-02 change or add files inside the existing ap_pipeline repository only. `delays.py` is a new module, not a new repository	feat.tasks (TSK-01, TSK-02 "Files in budget")	feat.tasks
P-4	Remote	`origin` is `https://github.com/arghya05/ap-pipeline-sample.git`; `main` and `dev` track `origin`	`.git/config`	file:ap_pipeline/.git/config
P-5	Branches	`main` and `dev` both point at commit `3fcbe1b`. The feature branch `feature/story` was created from that commit, and `HEAD` is on `feature/story`	`.git/refs/heads/main`, `.git/refs/heads/dev`, `.git/HEAD`, `.git/logs/refs/heads/feature/story` ("branch: Created from HEAD"); tool view: "branch feature/story created from dev at 3fcbe1b"	file:ap_pipeline/.git/HEAD; tool:repo-access
P-6	Branch convention	The branch follows `feature/<story>`, alongside `main` (released) and `dev` (integration)	Repository memory; org.engineering_stack; feat.tasks section 2	file:ap_pipeline/CLAUDE.md; org.engineering_stack; feat.tasks
P-7	Write access	**Pass.** The agent identity has write access, and the working tree is clean	Tool view: "the agent identity has write access (git status clean)"	tool:repo-access
P-8	Push	**Spine only.** Pushing, merging and other remote operations are run by the spine, not by the agent (the step hook denied them). `feature/story` has no upstream entry in `.git/config` yet. The spine sets it at push time (DE-10)	`.git/config` has `[branch]` entries only for `main` and `dev`	file:ap_pipeline/.git/config
P-9	CI command	`/usr/bin/python3 ci.py`: compile, unit tests, coverage `COVERAGE_MIN = 90.0`, and static checks (no `print`/`eval`/`exec`, no bare `except`, lines at most 140 characters)	`ci.py`	file:ap_pipeline/ci.py; org.engineering_stack

#	Check	Result	Evidence	Source
P-10	Current package version	`__version__ = "0.4.0"`, which matches the "ap_pipeline `0.4.0` today" baseline in feat.tasks (TSK-01 owns the version step)	`src/ap_pipeline/__init__.py`	file:ap_pipeline/src/ap_pipeline/__init__.py; feat.tasks
P-11	Existing schedule (read only; TSK-01 must not change it)	`schedule.json`: `daily`, `02:00`, `Asia/Kolkata`	`src/ap_pipeline/schedule.json`	file:ap_pipeline/src/ap_pipeline/schedule.json; feat.tasks (TSK-01 "Must not change")

3. Outcome

Prerequisite	Status
Databricks	Not applicable: none is used (org.engineering_stack; tool:repo-access)
Repository ap_pipeline	Available locally, with remote `ap-pipeline-sample` (tool:repo-access; file:ap_pipeline/.git/config)
Repository creation	Not needed (feat.tasks)
Write access	Yes, on `feature/story`, created from `dev` at `3fcbe1b`; working tree clean (tool:repo-access)

The Data Engineering lane can start TSK-01 on `feature/story`. TSK-02 follows after TSK-01 (feat.tasks section 1).

4. Observations (non-blocking)

ID	Observation	Effect	Source
RA-1	`ci.py` imports the third-party package `coverage`. The repository memory says "Python 3.9 (standard library)" for the code. This step could not confirm that `coverage` is installed for `/usr/bin/python3`: shell commands are denied for DE-01, and a file search under the Command Line Tools Python framework found no `site-packages/coverage`	If `coverage` is missing, CI fails before the tests run. This is a CI-tooling prerequisite, not a product dependency	file:ap_pipeline/ci.py; file:ap_pipeline/CLAUDE.md
RA-2	The branch is named `feature/story`, not `feature/STORY-01`. Both Data Engineering tasks (TSK-01, TSK-02) will share this one branch	Commit messages still start with the story ID (`STORY-01`, `STORY-02`)	tool:repo-access; feat.tasks (section 2); record:memory

5. Open questions (all non-blocking)

ID	Question	Interim answer	Owner (proposed)
RAQ-1	Is the `coverage` package installed for `/usr/bin/python3`, the interpreter that runs `ci.py`? (RA-1)	Assume yes. The first CI run in DE-09 or DE-10 confirms it	Data engineering lead / spine

ID	Question	Interim answer	Owner (proposed)
RAQ-2	Should TSK-01 and TSK-02 share `feature/story`, or should each story get its own `feature/STORY-0n` branch? (RA-2)	Share `feature/story`, as the spine created it	Data engineering lead

Tool results (recorded by the spine from the real tools)

[Tool result] [ap_pipeline] Branch feature/story at 3fcbe1b; write access confirmed